MODULE 1 – HTML & CSS FUNDAMENTALS

TABLE OF CONTENTS

PART 1 – HTML FUNDAMENTALS

1. Introduction to Web Development
2. Introduction to HTML
3. Structure of an HTML Document
4. HTML Tags, Elements and Attributes
5. HTML Headings and Paragraphs
6. HTML Text Formatting
7. HTML Links and Images
8. HTML Lists
9. HTML Tables
10. HTML Forms
11. HTML Input Types
12. Semantic HTML
13. Block-Level and Inline Elements
14. Void Elements
15. HTML5 Features

PART 2 – CSS FUNDAMENTALS

16. Introduction to CSS
17. Types of CSS
18. CSS Syntax
19. CSS Selectors
20. CSS Colors and Backgrounds
21. CSS Fonts and Text Properties
22. CSS Units
23. CSS Box Model
24. Margin, Padding and Border
25. CSS Display Properties
26. CSS Position Properties
27. CSS Flexbox
28. CSS Grid

29. Pseudo-classes and Pseudo-elements
30. Borders, Shadows and Gradients
31. CSS Transitions and Transformations
32. Responsive Web Design
33. CSS Media Queries
34. CSS Specificity and Inheritance

PART 1 – HTML FUNDAMENTALS

1. INTRODUCTION TO WEB DEVELOPMENT

Web development is the process of creating websites and web applications that users access through web browsers.

Web development is generally divided into three main technologies on the frontend.

1.1 Frontend Technologies

Technology	Purpose
HTML	Defines the structure of a webpage.
CSS	Controls the appearance and layout of a webpage.
JavaScript	Adds functionality and interactivity.

1.2 Understanding with an Example

Consider a student registration webpage.

- HTML creates the name, email and password fields.
- CSS adds colors, borders, spacing and alignment.
- JavaScript validates user input and handles interactions.

1.3 How a Website Works

1. The user enters a website URL in the browser.
2. The browser sends a request to the server.
3. The server returns the requested resources.
4. The browser processes HTML, CSS and JavaScript.
5. The webpage is displayed to the user.

---

2. INTRODUCTION TO HTML

HTML stands for **HyperText Markup Language**.

HTML is the standard markup language used to create and structure web pages.

2.1 Why Do We Use HTML?

1. To create webpage structures.
2. To display text and images.
3. To create hyperlinks.
4. To create tables and forms.
5. To organize webpage content.
6. To embed audio and video.

2.2 Features of HTML

- Easy to learn and understand.
- Platform independent.
- Supported by modern web browsers.
- Supports multimedia.
- Supports hyperlinks.
- Provides semantic elements.
- Works with CSS and JavaScript.

---

3. STRUCTURE OF AN HTML DOCUMENT

Every HTML document follows a basic structure.

3.1 Basic HTML Program

```
None
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>My First Webpage</title>
</head>

<body>

    <h1>Welcome to HTML</h1>
    <p>This is my first webpage.</p>

</body>
</html>
```

3.2 Explanation of HTML Structure

Element	Explanation
<!DOCTYPE html>	Declares that the document uses HTML5.
<html>	Root element of the HTML document.
<head>	Contains metadata and document information.
<title>	Defines the browser tab title.
<meta>	Provides metadata about the webpage.
<body>	Contains the visible webpage content.
<h1>	Defines the main heading.
<p>	Defines a paragraph.

3.3 Important Meta Tags

Character Encoding

```
None
<meta charset="UTF-8">
```

Specifies the character encoding used by the webpage.

Viewport

```
None
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

Helps the webpage adapt to different device screen widths.

---

4. HTML TAGS, ELEMENTS AND ATTRIBUTES

4.1 HTML Tags

Tags are keywords enclosed within angle brackets.

Example:

```
None
<p>Hello World</p>
```

Here:

- `<p>` is the opening tag.
- `</p>` is the closing tag.

4.2 HTML Elements

An HTML element generally consists of an opening tag, content and a closing tag.

Example:

```
None
<h1>HTML Fundamentals</h1>
```

4.3 HTML Attributes

Attributes provide additional information about HTML elements.

Example:

```
None
<a href="https://www.google.com">Google</a>
```

Explanation:

- `<a>` – Anchor element.
- `href` – Attribute name.
- URL – Attribute value.
- Google – Link text.

4.4 Common HTML Attributes

Attribute	Purpose
id	Identifies a unique element.
class	Groups elements for styling.
href	Specifies a hyperlink destination.
src	Specifies a resource location.
alt	Provides alternative text for images.
style	Applies inline CSS.
title	Provides additional advisory information.
name	Identifies a form control during submission.
value	Specifies an input's value.
type	Specifies an element or input type.

5. HTML HEADINGS AND PARAGRAPHS

HTML provides six heading levels.

```
None
<h1>Main Heading</h1>
<h2>Sub Heading</h2>
<h3>Section Heading</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>
```

Paragraph Tag

The `<p>` element defines a paragraph.

```
None
<p>HTML is used to create webpage structures.</p>
```

Line Break

The `<br>` element inserts a line break.

```
None
<p>Hello<br>Welcome to HTML</p>
```

Horizontal Line

The `<hr>` element represents a thematic break.

```
None
<h2>HTML</h2>
<hr>
<h2>CSS</h2>
```

---

6. HTML TEXT FORMATTING

HTML provides different elements to format and emphasize text.

Element	Purpose
`<b>`	Displays bold text.
`<strong>`	Indicates important text.
`<i>`	Displays text in an alternate voice or mood, commonly italic.
`<em>`	Emphasizes text.
`<u>`	Marks annotated text, commonly underlined.
`<mark>`	Highlights text.
`<del>`	Represents deleted text.
`<ins>`	Represents inserted text.
`<sub>`	Displays subscript text.
`<sup>`	Displays superscript text.

Example

```
None
<p><strong>Important:</strong> Learn HTML.</p>

<p><em>Welcome to CSS.</em></p>

<p>Water Formula: H<sub>2</sub>O</p>

<p>Mathematics: X<sup>2</sup></p>

<p><mark>Highlighted Text</mark></p>
```

---

7. HTML LINKS AND IMAGES

7.1 Hyperlinks

The <a> element creates hyperlinks.

Syntax:

```
None
<a href="URL">Link Text</a>
```

Example:

```
HTML
<a href="https://www.google.com">Visit Google</a>
```

Opening a Link in a New Tab

```
None
<a href="https://www.google.com"
   target="_blank"
   rel="noopener noreferrer">
   Open Google
</a>
```

Types of Links

1. External links – Link to another website.
2. Internal links – Link to another page in the same website.
3. Anchor links – Navigate to a section within a page.
4. Email links – Open an email application.

Example:

```
None
<a href="about.html">About Us</a>

<a href="#contact">Contact Section</a>

<a href="mailto:example@gmail.com">Send Email</a>
```

7.2 Images

The `<img>` element displays images on a webpage.

Syntax:

```
None
<img src="image.jpg" alt="Description">
```

Example:

```
None
<img src="student.jpg"
     alt="Student Image"
     width="300"
     height="200">
```

Image Attributes

Attribute	Purpose
src	Specifies image location.
alt	Provides alternative text.
width	Defines image width.
height	Defines image height.

The `<img>` element is a void element and does not require a closing tag.

---

8. HTML LISTS

HTML provides three major types of lists.

8.1 Ordered List

An ordered list displays items in numbered order.

```
None
<ol>
    <li>HTML</li>
    <li>CSS</li>
    <li>JavaScript</li>
</ol>
```

8.2 Unordered List

An unordered list displays items using bullets.

```
None
<ul>
    <li>Java</li>
    <li>C#</li>
    <li>Python</li>
</ul>
```

8.3 Description List

A description list displays terms and their descriptions.

```
None
<dl>
    <dt>HTML</dt>
    <dd>Defines webpage structure.</dd>

    <dt>CSS</dt>
```

```
    <dd>Controls webpage appearance.</dd>
</dl>
```

List Elements

Element	Purpose
ol	Ordered list.
ul	Unordered list.
li	List item.
dl	Description list.
dt	Description term.
dd	Description details.

---

9. HTML TABLES

HTML tables organize information into rows and columns.

9.1 Important Table Elements

Element	Purpose
table	Creates a table.
tr	Creates a row.
th	Defines a header cell.
td	Defines a data cell.
thead	Groups header rows.
tbody	Groups body rows.
tfoot	Groups footer rows.

caption	Defines a table title.

9.2 Example

```
None
<table border="1">

    <caption>Student Details</caption>

    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Department</th>
        </tr>
    </thead>

    <tbody>
        <tr>
            <td>101</td>
            <td>Arun</td>
            <td>CSE</td>
        </tr>

        <tr>
            <td>102</td>
            <td>Priya</td>
            <td>ECE</td>
        </tr>
    </tbody>

</table>
```

9.3 Table Attributes

colspan: Combines multiple columns.

```
None
<td colspan="2">Combined Columns</td>
```

rowspan: Combines multiple rows.

```
None
<td rowspan="2">CSE</td>
```

Note: CSS is generally preferred over the old border attribute for table styling.

---

10. HTML FORMS

Forms are used to collect information from users.

Examples:

- Login forms.
- Registration forms.
- Contact forms.
- Feedback forms.
- Application forms.

10.1 Important Form Elements

Element	Purpose
form	Defines a form.
label	Defines an input label.
input	Accepts user input.
textarea	Accepts multiline text.
select	Creates a dropdown.
option	Defines dropdown options.
button	Creates a button.
fieldset	Groups related form controls.

legend	Defines a caption for a fieldset.

10.2 Form Example

```
None
<form action="/register" method="post">

    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required>

    <br><br>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>

    <br><br>

    <label for="password">Password:</label>
    <input type="password" id="password"
           name="password" required>

    <br><br>

    <button type="submit">Register</button>

</form>
```

10.3 Form Attributes

Attribute	Purpose
action	Specifies where form data is submitted.
method	Specifies the HTTP request method.
required	Makes an input mandatory.
placeholder	Displays an input hint.
value	Specifies an input value.

name	Defines the submitted field name.
autocomplete	Controls browser autofill behavior.
readonly	Makes a field non-editable but still submittable.
disabled	Disables a form control.
maxlength	Specifies the maximum text length.
minlength	Specifies the minimum text length.

10.4 GET and POST Methods

GET Method

- Commonly used to retrieve information.
- Parameters are generally included in the URL.
- Useful for search forms and filters.

POST Method

- Sends submitted data in the request body.
- Commonly used for registration and data submission.
- Does not automatically encrypt data; HTTPS is still necessary.

---

11. HTML INPUT TYPES

The input element supports different types of user input.

Type	Purpose
text	Accepts text.
password	Accepts password input.
email	Accepts email addresses.
number	Accepts numeric values.
date	Selects a date.

radio	Selects one option from a group.
checkbox	Selects one or more options.
file	Uploads a file.
tel	Accepts telephone numbers.
url	Accepts a URL.
time	Selects a time.
color	Selects a color.
submit	Submits a form.
reset	Resets form controls.

Example

None

```
<form>

    Name:
    <input type="text"><br><br>

    Password:
    <input type="password"><br><br>

    Gender:
    <input type="radio" name="gender" value="male"> Male
    <input type="radio" name="gender" value="female"> Female

    <br><br>

    Skills:
    <input type="checkbox" name="skill" value="html"> HTML
    <input type="checkbox" name="skill" value="css"> CSS

    <br><br>
```

```
    Date of Birth:
    <input type="date"><br><br>

    <input type="submit" value="Submit">

</form>
```

---

12. SEMANTIC HTML

Semantic HTML elements clearly describe their meaning and purpose.

12.1 Important Semantic Elements

Element	Purpose
header	Defines introductory content.
nav	Defines navigation links.
main	Defines the main page content.
section	Defines a thematic section.
article	Defines independent content.
aside	Defines supplementary content.
footer	Defines footer information.

12.2 Example

```
None
<!DOCTYPE html>
<html>
<head>
    <title>Semantic HTML</title>
</head>
```

```
<body>

    <header>
        <h1>My Website</h1>
    </header>

    <nav>
        <a href="#home">Home</a>
        <a href="#about">About</a>
    </nav>

    <main>

        <section>
            <h2>About Us</h2>
            <p>Welcome to our website.</p>
        </section>

        <article>
            <h2>Latest News</h2>
            <p>Website development updates.</p>
        </article>

    </main>

    <footer>
        <p>Copyright 2026</p>
    </footer>

</body>
</html>
```

12.3 Advantages of Semantic HTML

1. Improves code readability.
2. Helps search engines understand page structure.
3. Improves accessibility.

4. Makes website maintenance easier.
5. Gives meaningful structure to webpage content.

---

13. BLOCK-LEVEL AND INLINE ELEMENTS

HTML elements commonly have different default display behaviors.

Block-Level Elements

- Start on a new line.
- Generally occupy the available width.
- Can contain other appropriate elements.

Examples: div, p, h1, section, article.

Inline Elements

- Flow within surrounding text.
- Occupy the space required by their content.
- Width and height do not behave like normal block elements.

Examples: span, a, strong, em.

Difference

Block	Inline
Starts on a new line by default.	Does not normally start on a new line.
Usually occupies available width.	Occupies content width.
Width and height can generally be set.	Width and height do not normally control its layout like a block.
Example: div	Example: span

---

14. VOID ELEMENTS

Void elements are HTML elements that do not have closing tags and cannot contain child elements.

Examples:

Element	Purpose
br	Inserts a line break.
hr	Represents a thematic break.
img	Displays an image.
input	Creates an input control.
meta	Provides metadata.
link	Connects external resources.
area	Defines an image-map area.
source	Specifies media resources.

Example:

```
None
<img src="photo.jpg" alt="Photo">

<br>

<hr>

<input type="text">
```

---

15. HTML5 FEATURES

HTML5 is the modern HTML standard.

Important features include:

1. Semantic elements such as header, nav, main and footer.
2. Native audio and video elements.
3. Improved form input types.
4. Canvas for drawing graphics.
5. SVG support.
6. Improved browser APIs.
7. Better support for structured web content.

Audio Example

```
None
<audio controls>
    <source src="audio.mp3" type="audio/mpeg">
</audio>
```

Video Example

```
None
<video width="400" controls>
    <source src="video.mp4" type="video/mp4">
</video>
```

PART 2 – CSS FUNDAMENTALS

16. INTRODUCTION TO CSS

CSS stands for **Cascading Style Sheets**.

CSS is used to control the presentation and appearance of HTML elements.

Why Do We Use CSS?

1. To change text colors.
2. To add backgrounds.
3. To control font styles and sizes.
4. To manage spacing and alignment.
5. To create webpage layouts.
6. To make websites responsive.

7. To improve user experience.

Example

HTML:

```
None
<h1>Welcome to CSS</h1>
```

CSS:

```
None
h1 {
    color: blue;
    font-size: 30px;
}
```

Output: The heading appears in blue with a font size of 30 pixels.

---

17. TYPES OF CSS

There are three main ways to apply CSS.

17.1 Inline CSS

CSS is written directly inside an HTML element using the style attribute.

```
None
<h1 style="color: red;">Welcome</h1>
```

Advantages:

- Quick for small demonstrations.
- Styles one specific element.

Disadvantages:

- Difficult to maintain.
- Not reusable.
- Mixes content and presentation.

17.2 Internal CSS

CSS is written inside the style element in the HTML document.

```
None
<head>

<style>

h1 {
    color: blue;
    font-size: 30px;
}

</style>

</head>
```

Advantages:

- Suitable for single-page examples.
- Keeps CSS in one place within the document.

Disadvantages:

- Not easily reusable across multiple pages.

17.3 External CSS

CSS is written in a separate file with the `.css` extension.

HTML:

```
None
<link rel="stylesheet" href="style.css">
```

style.css:

```
None
h1 {
    color: green;
    font-size: 30px;
}
```

Advantages:

- Reusable across pages.
- Easier maintenance.
- Better separation of concerns.
- Preferred for larger applications.

Comparison

Inline CSS	Internal CSS	External CSS
Inside an element.	Inside style element.	Separate CSS file.
Applies directly to an element.	Applies within the document.	Can apply across multiple documents.
Difficult to maintain at scale.	Useful for single pages.	Preferred for larger projects.

---

18. CSS SYNTAX

CSS follows a simple syntax.

```
None
selector {
    property: value;
}
```

Example:

```
None
h1 {
    color: blue;
    font-size: 30px;
    text-align: center;
}
```

Explanation:

- Selector: h1
- Property: color
- Value: blue
- Declaration: color: blue;
- Declaration block: All declarations between the braces.

---

19. CSS SELECTORS

CSS selectors identify HTML elements that should receive styles.

19.1 Element Selector

Selects elements by tag name.

```
None
p {
    color: red;
}
```

19.2 ID Selector

Selects an element using its ID.

```
None
#heading {
    color: blue;
}
```

HTML:

```
None
<h1 id="heading">Welcome</h1>
```

19.3 Class Selector

Selects elements using their class name.

```
None
.box {
    background-color: yellow;
}
```

HTML:

```
None
<div class="box">Hello</div>
```

19.4 Universal Selector

Selects all elements.

```
None
* {
    margin: 0;
    padding: 0;
}
```

19.5 Group Selector

Applies the same styles to multiple selectors.

```
None
h1, h2, p {
    color: green;
}
```

19.6 Descendant Selector

Selects elements located anywhere inside another element.

```
None
div p {
    color: red;
}
```

19.7 Child Selector

Selects direct child elements.

```
None
div > p {
    color: blue;
}
```

Selector Summary

Selector	Syntax
Element	p
ID	#heading
Class	.container
Universal	*
Group	h1, p
Descendant	div p
Child	div > p

---

20. CSS COLORS AND BACKGROUNDS

CSS supports different color formats.

Color Formats

1. Named colors.
2. HEX.
3. RGB.
4. RGBA.
5. HSL.

Example:

```
None
h1 {
    color: red;
}

p {
    color: #0000FF;
}

div {
    background-color: rgb(255, 255, 0);
}
```

Background Properties

Property	Purpose
background-color	Sets background color.
background-image	Sets background image.
background-repeat	Controls image repetition.
background-position	Sets image position.
background-size	Controls image size.

Example:

```
None
body {
```

```
    background-color: lightblue;
}

.container {
    background-image: url("image.jpg");
    background-repeat: no-repeat;
    background-position: center;
    background-size: cover;
}
```

---

21. CSS FONTS AND TEXT PROPERTIES

Font Properties

Property	Purpose
font-family	Defines font type.
font-size	Defines font size.
font-weight	Defines font thickness.
font-style	Defines normal or italic style.
font-variant	Controls small-cap styling and related variants.

Text Properties

Property	Purpose
text-align	Aligns text.
text-decoration	Adds or removes text decoration.
text-transform	Changes text capitalization.
text-indent	Sets first-line indentation.
line-height	Controls line spacing.

letter-spacing	Controls spacing between characters.

Example:

```
None
p {
    font-family: Arial, sans-serif;
    font-size: 16px;
    font-weight: bold;
    text-align: center;
    text-decoration: none;
    line-height: 1.5;
}
```

---

22. CSS UNITS

CSS uses different units to define dimensions, spacing and font sizes.

Absolute Unit

px: CSS pixel.

Example:

```
None
div {
    width: 200px;
}
```

Relative Units

Unit	Meaning
%	Relative to a relevant parent or containing dimension.
em	Relative to the relevant element font-size context.

rem	Relative to the root element font size.
vw	1% of viewport width.
vh	1% of viewport height.
fr	Fraction of available space in CSS Grid.

Example:

```
None
.container {
    width: 80%;
    font-size: 1rem;
    padding: 2em;
    height: 50vh;
}
```

Difference Between em and rem

- em is relative to the relevant font-size context.
- rem is relative to the root HTML element's font size.

---

23. CSS BOX MODEL

The CSS Box Model describes how every HTML element occupies space.

It consists of four components:

1. Content.
2. Padding.
3. Border.
4. Margin.

23.1 Content

The actual text, image or other content inside the element.

23.2 Padding

Space between the content and border.

23.3 Border

The boundary surrounding the padding and content.

23.4 Margin

Space outside the border.

Example

```
None
.box {
    width: 200px;
    padding: 20px;
    border: 5px solid black;
    margin: 10px;
}
```

Box Sizing

By default, CSS uses content-box sizing.

```
None
box-sizing: border-box;
```

With border-box, the specified width and height include content, padding and border.

Common global reset:

```
None
*,
*::before,
*::after {
    box-sizing: border-box;
}
```

---

24. MARGIN, PADDING AND BORDER

Margin

Controls space outside an element's border.

```
None
div {
    margin: 20px;
}
```

Padding

Controls space inside an element's border.

```
None
div {
    padding: 20px;
}
```

Border

Defines the boundary of an element.

```
None
div {
    border: 2px solid black;
}
```

Comparison

Margin	Padding	Border
Outside the border.	Inside the border.	Surrounds content and padding.
Controls external spacing.	Controls internal spacing.	Defines the boundary.

Background does not extend into margin.	Background extends behind padding.	Can have its own style and color.

---

25. CSS DISPLAY PROPERTIES

The display property controls how an element participates in layout.

Common Values

Value	Purpose
block	Creates a block-level box.
inline	Creates an inline box.
inline-block	Flows inline but supports width and height.
none	Removes the element from layout.
flex	Creates a flex container.
grid	Creates a grid container.

Example:

```
None
div {
    display: block;
}

span {
    display: inline;
}

button {
    display: inline-block;
}

.container {
```

```
    display: flex;
}
```

Difference: display and visibility

display: none

- Element is removed from the layout.
- No space is reserved for it.

visibility: hidden

- Element is invisible.
- Its layout space is preserved.

---

26. CSS POSITION PROPERTIES

The position property controls how an element is positioned.

Types of Positioning

Position	Explanation
static	Default positioning in normal document flow.
relative	Remains in normal flow and can be visually offset.
absolute	Removed from normal flow and positioned relative to its containing block.
fixed	Typically positioned relative to the viewport.
sticky	Switches between relative-like and stuck positioning during scrolling.

Example

```
None
.container {
    position: relative;
```

```
}

.box {
    position: absolute;
    top: 10px;
    right: 20px;
}
```

The absolutely positioned box is positioned relative to its containing block, established here by the positioned container.

Z-Index

The z-index property influences the stacking order of elements.

```
None
.box {
    position: absolute;
    z-index: 10;
}
```

A higher z-index generally places an element above a lower one within the relevant stacking context.

---

27. CSS FLEXBOX

Flexbox is a one-dimensional layout system used to arrange elements in rows or columns.

Why Do We Use Flexbox?

- To align elements.
- To distribute available space.
- To create navigation bars.
- To center content.
- To create flexible layouts.

Important Flexbox Properties

Property	Purpose
display	Defines a flex container.
flex-direction	Defines row or column direction.
justify-content	Aligns items along the main axis.
align-items	Aligns items along the cross axis.
flex-wrap	Controls wrapping.
gap	Defines spacing between items.
flex	Controls item growth, shrinkage and basis.

Example

```
None
<div class="container">
    <div class="box">One</div>
    <div class="box">Two</div>
    <div class="box">Three</div>
</div>
```

CSS:

```
None
.container {
    display: flex;
    flex-direction: row;
    justify-content: center;
    align-items: center;
    gap: 20px;
}

.box {
    padding: 20px;
    background-color: lightblue;
```

```
}
```

Flex Direction

```
None
flex-direction: row;
flex-direction: column;
flex-direction: row-reverse;
flex-direction: column-reverse;
```

Justify Content

```
None
justify-content: center;
justify-content: flex-start;
justify-content: flex-end;
justify-content: space-between;
justify-content: space-around;
justify-content: space-evenly;
```

---

28. CSS GRID

CSS Grid is a two-dimensional layout system used to arrange elements in rows and columns.

Important Grid Properties

Property	Purpose
display	Creates a grid container.
grid-template-columns	Defines column tracks.
grid-template-rows	Defines row tracks.
gap	Defines spacing between rows and columns.

grid-column	Controls an item's column placement.
grid-row	Controls an item's row placement.

Example

```
None
<div class="container">
    <div>One</div>
    <div>Two</div>
    <div>Three</div>
    <div>Four</div>
</div>
```

CSS:

```
None
.container {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 20px;
}

.container div {
    background-color: lightblue;
    padding: 20px;
}
```

Responsive Grid

```
None
.container {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
    gap: 20px;
}
```

This allows the browser to fit an appropriate number of columns based on available space.

Flexbox vs Grid

Flexbox	Grid
One-dimensional layout.	Two-dimensional layout.
Works mainly with a row or column.	Works with rows and columns together.
Useful for navigation bars.	Useful for page layouts.
Useful for aligning components.	Useful for card grids and galleries.

---

29. PSEUDO-CLASSES AND PSEUDO-ELEMENTS

29.1 Pseudo-classes

Pseudo-classes select elements based on their state or position.

Examples:

Pseudo-class	Purpose
	When the pointer is over an element.
	When an element has focus.
	While an element is being activated.
	Selects the first child.
	Selects the last child.
()	Selects elements based on their position.
	Selects checked controls.
	Selects disabled controls.

Example:

```
None
button:hover {
    background-color: blue;
}

input:focus {
    border-color: green;
}

li:first-child {
    font-weight: bold;
}
```

29.2 Pseudo-elements

Pseudo-elements style a specific part of an element or create generated content.

Examples:

Pseudo-element	Purpose
::before	Inserts generated content before an element's content.
::after	Inserts generated content after an element's content.
::first-letter	Styles the first letter.
::first-line	Styles the first line.
::selection	Styles selected text.

Example:

```
None
p::first-letter {
    font-size: 30px;
}
```

```
h1::after {
    content: "!";
}
```

Difference

- Pseudo-class selects an element in a particular state.
- Pseudo-element targets a specific part of an element or generated content.

---

30. BORDERS, SHADOWS AND GRADIENTS

Border

None
```
.box {
    border: 2px solid black;
    border-radius: 10px;
}
```

Box Shadow

None
```
.card {
    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.2);
}
```

Text Shadow

None
```
h1 {
    text-shadow: 2px 2px 4px gray;
}
```

Linear Gradient

```
None
body {
    background: linear-gradient(to right, blue, purple);
}
```

Radial Gradient

```
None
.box {
    background: radial-gradient(circle, white, blue);
}
```

---

31. CSS TRANSITIONS AND TRANSFORMATIONS

31.1 Transitions

Transitions create smooth changes between CSS property values.

Example:

```
None
button {
    background-color: blue;
    transition: background-color 0.3s ease;
}

button:hover {
    background-color: green;
}
```

Important properties:

- transition-property
- transition-duration
- transition-timing-function
- transition-delay

31.2 Transformations

The transform property changes the visual position, size or rotation of an element.

Translate

```
None
transform: translate(20px, 10px);
```

Moves an element.

Rotate

```
None
transform: rotate(45deg);
```

Rotates an element.

Scale

```
None
transform: scale(1.2);
```

Increases or decreases the element's visual size.

Skew

```
None
transform: skew(10deg);
```

Skews an element.

---

32. RESPONSIVE WEB DESIGN

Responsive Web Design is an approach that makes websites adapt to different screen sizes and devices.

Why Is Responsive Design Important?

1. Supports mobile devices.
2. Improves user experience.
3. Avoids unnecessary horizontal scrolling.
4. Supports tablets and desktops.
5. Makes layouts flexible.

Common Techniques

- Flexible layouts.
- Relative units.
- Responsive images.
- Flexbox.
- CSS Grid.
- Media queries.

---

33. CSS MEDIA QUERIES

Media queries apply CSS styles based on device or viewport conditions.

Syntax

```
None
@media (condition) {

    selector {
        property: value;
    }

}
```

Example

None

```
.container {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
}

@media (max-width: 768px) {

    .container {
        grid-template-columns: 1fr;
    }

}
```

Explanation:

- On wider screens, the layout uses three columns.
- At viewport widths of 768px or below, it changes to one column.

Common Media Query Conditions

None

```
@media (max-width: 768px) {
    /* Tablet and smaller layout */
}

@media (max-width: 480px) {
    /* Smaller mobile layout */
}

@media (min-width: 1024px) {
    /* Larger screen layout */
}
```

Breakpoints should be selected based on when the content needs a layout change.

---

34. CSS SPECIFICITY AND INHERITANCE

34.1 CSS Specificity

Specificity determines the priority of competing CSS selectors within the cascade.

Simplified priority:

1. ID selectors.
2. Class, attribute and pseudo-class selectors.
3. Element and pseudo-element selectors.

Example:

```
None
p {
    color: red;
}

.text {
    color: blue;
}

#message {
    color: green;
}
```

HTML:

```
None
<p class="text" id="message">
    Hello World
</p>
```

The text is green because the ID selector has higher specificity than the class and element selectors.

The cascade also considers origin, importance, cascade layers and source order.

34.2 CSS Inheritance

Inheritance allows certain CSS properties to pass from parent elements to child elements.

Example:

```
None
body {
    color: blue;
    font-family: Arial;
}
```

Text inside child elements generally inherits these properties unless another rule overrides them.

Common inherited properties:

- color
- font-family
- font-size
- line-height

Common non-inherited properties:

- margin
- padding
- border
- width
- height
